

Exception Handling and Multithreaded Programming:

**SIMATS**
ENGINEERING

SIMATS Online Programming Lab
JAVA PROGRAMMING

B THARUN KUMAR
192424356RC

QUESTIONS 8 / 8 completed

Counter-T

COUNTER-T

Write a Java program to create a thread-safe counter that can be accessed by multiple threads simultaneously.

PROGRAM EDITOR

Java

Run

Save

```
1 public class MyThread extends Thread {
2
3     static int counter = 0;
4     synchronized void incrementCounter() {
5         counter++;
6     }
7
8     public void run() {
9
10        for (int i = 1; i <= 5; i++) {
11            incrementCounter();
12        }
13    }
14
15    public static void main(String[] args) {
16        int numberOfThreads = 3;
17        MyThread t1 = new MyThread();
18        MyThread t2 = new MyThread();
19        MyThread t3 = new MyThread();
20        t1.start();
21        t2.start();
22    }
23 }
```

OUTPUT

```
Number of threads: 3
Final counter value: 15
```

INPUT

Your INPUT goes here!

**SIMATS**
ENGINEERING

SIMATS Online Programming Lab
JAVA PROGRAMMING

B THARUN KUMAR
192424356RC

QUESTIONS 8 / 8 completed

M-Thread

M-THREAD

Write a Java program to create multiple threads that perform some time-consuming task, such as calculating prime numbers or sorting an array, and compare their execution times.

PROGRAM EDITOR

Java

Run

Save

```
1 public class MyThread extends Thread {
2
3     int start;
4     int end;
5     long time;
6
7     MyThread(int start, int end) {
8         this.start = start;
9         this.end = end;
10    }
11
12    public void run() {
13
14        long startTime = System.nanoTime();
15
16        int count = 0;
17
18        for (int n = start; n <= end; n++) {
19
20            boolean prime = true;
21
22        }
23    }
24 }
```

OUTPUT

```
Thread for 1 to 100000 found 9592 prime numbers.
Thread for 100001 to 200000 found 8392 prime numbers.
Thread 1 execution time: 26434600 nanoseconds
```

INPUT

Your INPUT goes here!



SIMATS Online Programming Lab
JAVA PROGRAMMING

B THARUN KUMAR
192424356RC

QUESTIONS 8 / 8 completed

Pro-Consume

PRO-CONSUME
Write a Java program to create a producer-consumer scenario using multiple threads. The producer thread should add items to a queue, and the consumer thread should remove items from the queue and process them.

PROGRAM EDITOR

Java Run Save

```
1 import java.util.LinkedList;
2 import java.util.Queue;
3
4 public class MyThread {
5
6     static Queue<Integer> queue = new LinkedList<>();
7     static int MAX_SIZE = 5;
8
9     static class Producer extends Thread {
10
11         public void run() {
12
13             for (int i = 1; i <= 5; i++) {
14
15                 synchronized (queue) {
16
17                     while (queue.size() == MAX_SIZE) {
18                         try {
19                             queue.wait();
20                         } catch (InterruptedException e) {
21                             System.out.println("Producer interrupted");
```

OUTPUT
Number of items: 5

Producer produced: 1
Consumer consumed: 1

INPUT
Your INPUT goes here!



SIMATS Online Programming Lab
JAVA PROGRAMMING

B THARUN KUMAR
192424356RC

QUESTIONS 8 / 8 completed

Deadlock

DEADLOCK
Write a Java program to create a deadlock scenario using multiple threads.

PROGRAM EDITOR

Java Run Save

```
1 public class DeadlockExample {
2
3     static Object resource1 = new Object();
4     static Object resource2 = new Object();
5
6     public static void main(String[] args) {
7
8         Thread t1 = new Thread(() -> {
9             synchronized (resource1) {
10                 System.out.println("Thread 1 locked Resource 1");
11
12                 try {
13                     Thread.sleep(100);
14                 } catch (InterruptedException e) {
15                 }
16
17                 synchronized (resource2) {
18                     System.out.println("Thread 1 locked Resource 2");
19                 }
20             }
21         });
```

OUTPUT
Thread 1 locked Resource 1
Thread 2 locked Resource 2
Deadlock scenario created.

INPUT
Your INPUT goes here!

SIMATS Online Programming Lab

JAVA PROGRAMMING

B THARUN KUMAR

192424356RC

QUESTIONS 8 / 8 completed

Wait-Notify

WAIT-NOTIFY

Write a Java program to demonstrate how to use the "wait" and "notify" methods to implement a producer-consumer scenario.

PROGRAM EDITOR
Java
Run Save

```

1 public class ProducerConsumer {
2
3     static class SharedQueue {
4         int item;
5         boolean available = false;
6
7         synchronized void produce(int value) {
8             try {
9                 while (available) {
10                     wait();
11                 }
12
13                 item = value;
14                 available = true;
15
16                 System.out.println("Produced: " + item);
17                 notify();
18
19             } catch (InterruptedException e) {
20                 Thread.currentThread().interrupt();
21             }
22         }
23     }
24 }

```

OUTPUT

Produced: 1
Consumed: 1
Produced: 2
Consumed: 2

INPUT

Your INPUT goes here!

SIMATS Online Programming Lab

JAVA PROGRAMMING

B THARUN KUMAR

192424356RC

QUESTIONS 8 / 8 completed

Pool

POOL

Write a Java program to create a thread pool that can execute multiple threads concurrently.

PROGRAM EDITOR
Java
Run Save

```

1 import java.util.concurrent.ExecutorService;
2 import java.util.concurrent.Executors;
3
4 public class ThreadPoolExample {
5
6     public static void main(String[] args) {
7
8         ExecutorService pool = Executors.newFixedThreadPool(3);
9
10        for (int i = 1; i <= 6; i++) {
11            int task = i;
12
13            pool.execute(() -> {
14                System.out.println("Task " + task + " is executed by " +
15                    Thread.currentThread().getName());
16            });
17
18            try {
19                Thread.sleep(1000);
20            } catch (InterruptedException e) {
21                Thread.currentThread().interrupt();
22            }
23        }
24    }
25 }

```

OUTPUT

Task 1 is executed by pool-1-thread-1
Task 2 is executed by pool-1-thread-2
Task 3 is executed by pool-1-thread-3

INPUT

Your INPUT goes here!

SIMATS Online Programming Lab

JAVA PROGRAMMING

B THARUN KUMAR

192424356RC

QUESTIONS 8 / 8 completed

Join

JOIN

Write a Java program to demonstrate how to use the "join" method to wait for a thread to complete its execution before proceeding with the main thread.

PROGRAM EDITOR
Java
Run Save

```

1 public class JoinExample {
2
3     public static void main(String[] args) {
4
5         Thread t1 = new Thread(() -> {
6             for (int i = 1; i <= 5; i++) {
7                 System.out.println("Thread 1: " + i);
8                 try {
9                     Thread.sleep(500);
10                } catch (InterruptedException e) {
11                    Thread.currentThread().interrupt();
12                }
13            }
14        });
15
16        t1.start();
17
18        try {
19            t1.join();
20        } catch (InterruptedException e) {
21            Thread.currentThread().interrupt();
22        }
23    }
24 }

```

OUTPUT

Thread 1: 1
Thread 1: 2
Thread 1: 3
Thread 1: 4

INPUT

Your INPUT goes here!